# Trabajo Práctico Integrador: Del Problema al Deploy

## Alumno: Toto Ignacio

**Seminario de Actualización · Tecnicatura Superior en Ciencia de Datos e IA · IFTS N.º 18**
**Docente: Cynthia M. Villagra · Ciclo lectivo 2026 · Segundo cuatrimestre**

# Contenido

El problema ..... 3
¿Qué problema querés resolver y para quién? ..... 3
¿Por qué un modelo de IA y no unas reglas escritas a mano? ..... 3
El modelo ..... 4
¿Qué modelos consideraste y por qué te quedaste con uno? ..... 4
¿Dónde se equivoca? ..... 4
Tu app, ¿está entrenando o infiriendo? ..... 5
Dónde corre cada cosa ..... 6
¿Dónde está guardado tu código? ..... 6
¿Dónde se ejecuta el código? ..... 6
¿Dónde están los pesos del modelo? ..... 6
¿Dónde se hace la inferencia? ..... 6
¿Usaste pipeline o InferenceClient en la app publicada? ..... 6
¿Qué pasaría si hubieras elegido la otra? ..... 6
El deploy ..... 6
¿Dónde deployaste y por qué ahí? ..... 6
¿Por qué tu app necesita un servidor que quede encendido? ..... 7
Lo que se rompió ..... 7
Límites de tu solución ..... 8
¿qué pasa si la usan 1000 personas a la vez? ..... 8
¿Si Hugging Face cambia las condiciones del plan gratuito (como ya pasó con los Spaces)? ..... 8
¿Qué cambiarías para que sea un producto real? ..... 8
Vocabulario ..... 8

# El problema

## ¿Qué problema querés resolver y para quién?

En la compañía estamos afrontando, luego de más de 20 años, la migración de un software de gestión de depósitos (WMS) desarrollado in-house hacia un software estándar de mercado, con el propósito de adoptar mejores prácticas logísticas y modernizar la infraestructura tecnológica.

Esta transición conlleva varios desafíos operativos y humanos, entre los que se destacan:

1. **Resistencia al cambio e incertidumbre operativa:** Operarios y supervisores acostumbrados a las mismas pantallas durante dos décadas enfrentan un cambio abrupto de flujos y pantallas.
2. **Saturación de los Key Users / referentes de soporte:** Durante las primeras semanas de arranque en vivo (Go-Live), los referentes quedan desbordados respondiendo dudas operativas básicas y repetitivas.
3. **Pérdida de tiempo y dispersión en la comunicación:** Los operarios envían la misma consulta a múltiples grupos de chat (WhatsApp, Teams, radio), generando desorganización y demoras.

Para mitigar este impacto, diseñamos una herramienta inteligente que analiza la duda del usuario y sugiere el área operativa y el canal de contacto adecuado, ordenando la comunicación y liberando a los referentes.

## ¿Por qué un modelo de IA y no unas reglas escritas a mano?

Un modelo de NLP Zero-Shot basado en Inferencia de Lenguaje Natural (NLI) es la solución adecuada porque interpreta el contexto de la consulta en lugar de buscar coincidencias literales de texto. El vocabulario del depósito presenta características que invalidan un enfoque tradicional de reglas if/else:

1. Jerga especializada y siglas operativas: PDA, PLU, HDR (Hoja de Ruta), Hueco, Slotting, Bultos, Pallets, Retrabajo.
2. Spanglish logístico y términos cruzados: Pickear, piking, reapro, repo, consolidar, andén.
3. Palabras omnipresentes que quiebran reglas sintácticas: Términos como PDA o pallets aparecen en prácticamente todas las preguntas.

Un sistema de reglas basado en if/else es incapaz de diferenciar si la consulta corresponde a Picking, Recepción, Expedición o Control de Inventario. El modelo Zero-Shot comprende la acción que el usuario quiere realizar en contexto y asigna las probabilidades correspondientes sin requerir entrenamiento previo.

# El modelo

## ¿Qué modelos consideraste y por qué te quedaste con uno?

**Modelo 1:** Recognai/bert-base-spanish-wwm-cased-xnli · Hugging Face

**Modelo 2:** Recognai/zeroshot_selectra_medium · Hugging Face

| **Modelo** | **bert-base-spanish-wwm-cased-xnli** | **Zero-shot SELECTRA** |
|---|---|---|
| Arquitectura | Usa la arquitectura **BERT** estándar (12 capas, 110 millones de parámetros). Entrena con *Masked Language* | Usa la arquitectura **ELECTRA** (~38-40 millones de parámetros en su versión medium). Usa la técnica de *Replaced Token Detection*. |
| Tarea | Zero-Shot Classification basada en NLI | Zero-Shot Classification basada en NLI |
| Etiquetas | Dinámicas**. Zero-Shot** no tiene etiquetas rígidas. Las puede definir el usuario en tiempo de ejecución | Dinámicas**. Zero-Shot** no tiene etiquetas rígidas. Las puede definir el usuario en tiempo de ejecución |
| Idiomas | Español | Español |
| Datos de Entrenamiento | Subconjunto en español del dataset académico **XNLI** | Subconjunto en español del dataset académico **XNLI** |
| Tamaño de los pesos | ~440 MB | ~163 MB |
| Consumo de memoria RAM | ~600 MB - 800 MB | < 300 MB (Apto Free Tier Streamlit 1 GB) |
| Decisión para el Deploy | Descartado (excede límites de memoria/latencia) | Seleccionado (ultraliviano, rápido y eficiente) |

## ¿Dónde se equivoca?

A partir de las pruebas ejecutadas en la notebook con 12 consultas reales del depósito, se identificaron los siguientes patrones y sesgos claros donde los modelos fallan:

1. Confusión sistemática entre **Preparación** y **Despacho de pedidos**

   - **Consulta**: En el Hueco no está el PLU que me dice la PDA que tengo que pickear. Como registro la incidencia?

   - **Respuesta**: Ambos modelos predijeron **Despacho de Pedidos** (**BERT**: 19.3%, **SELECTRA**: 30.4%), en lugar de **Preparación de Pedidos**.

   - **Consulta**: La repo de mercadería me envía a reponer un hueco de pikcing que está lleno.

   - **Respuesta**: Ambos modelos predijeron **Despacho de Pedidos** (**BERT**: 26.4%, **SELECTRA**; 49.2%).

**Conclusión**

La palabra picking o pickear no llega a guiar la respuesta de ninguno de los dos modelos hacía la preparación de pedidos, a pesar de que son términos ampliamente usado en ese sentido. Probablemente debido a que no son términos en español (lenguaje en el que están entrenado los modelos) y que a pesar de su amplio uso sigue siendo un término de nicho en la jerga de la logística.

2. Complejidad entre términos de significados similares

- **Consulta**: Estoy en el andén. Me falta cerrar el camión para que salga a la calle. Pero no se como se hace.
- **Respuesta:** Ambos modelos predijeron **Expedición de Camiones** (**BERT**: 31.1%, **SELECTRA**; 54.4%).
-
- **Consulta**: La PDA quiere consolidar pales de pollo con veneno para subirlos a un camión. Eso está mal.
- **Respuesta: BERT** predijo **Preparación de Pedidos** (23.8%) y **Expedición de Camiones** en segundo lugar (19.5%), mientras que **SELECTRA** predijo **Expedición de camiones** (68.7%).

**Conclusión**

Para un modelo de lenguaje general pre-entrenado, **Expedir** y **Despachar** son sinónimos casi exactos. Al no haber un entrenamiento supervisado con ejemplos específicos de la empresa, el modelo no puede diferenciar entre estas dos sub-áreas logísticas

3. Peso de los términos

- **Consulta**: El plan quedó chico. Necesito que envien más volumen para replnificar. Sino van a quedar los camiones por la mitad.
- **Respuesta:** Ambos modelos coinciden en **Expedición de Camiones** como primera respuesta y **Despacho de Pedidos** como segunda.

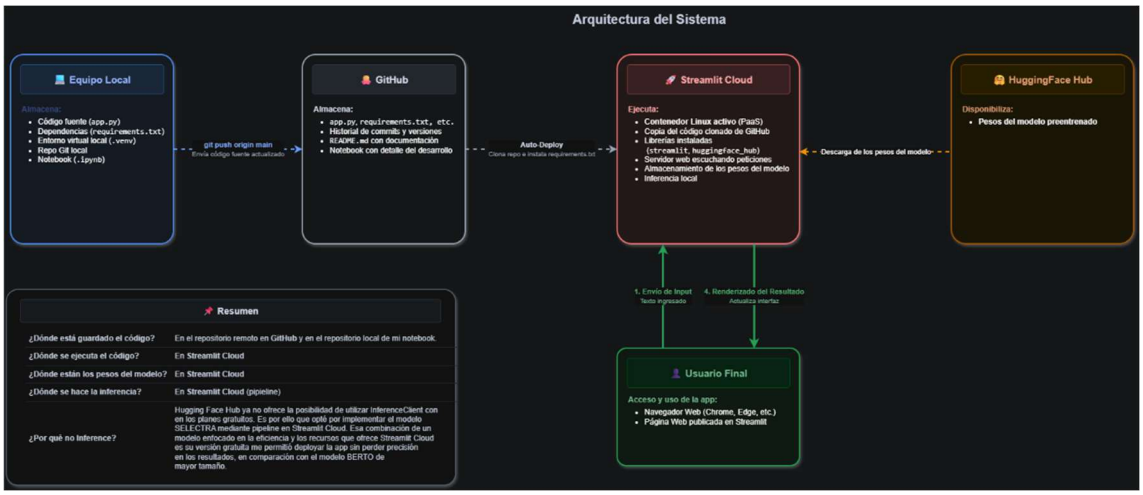
**Conclusión**

Los modelos no llegan a relacionar fuertemente el **plan** con la acción de planificar. Priorizan posiblemente la mención a los camiones y la vinculan a la expedición y despacho.

## Tu app, ¿está entrenando o infiriendo?

Está infiriendo. Ambos modelos evaluados ya fueron pre-entrenados y sus pesos están fijados. La app envía al modelo el texto para clasificarlo mediante Zero-Shot Classificaction con NLI.

# Dónde corre cada cosa




### ¿Dónde está guardado tu código?

En el repositorio remoto en GitHub y en el repositorio local de desarrollo.

### ¿Dónde se ejecuta el código?

En el servidor PaaS de Streamlit Community Cloud (y localmente en la PC durante el desarrollo).

### ¿Dónde están los pesos del modelo?

En el repositorio de Hugging Face Hub (desde donde se descargan a la memoria del contenedor de Streamlit al inicializar la app).

### ¿Dónde se hace la inferencia?

En la CPU del contenedor de Streamlit Cloud mediante la librería transformers y la clase pipeline, utilizando el modelo ligero SELECTRA-Medium (~163 MB).

### ¿Usaste pipeline o InferenceClient en la app publicada?

Utilicé pipeline de la librería transformers, ejecutado de manera local en el contenedor de Streamlit Cloud con el modelo ligero Recognai/zeroshot_selectra_medium (~163 MB).

### ¿Qué pasaría si hubieras elegido la otra?

Si hubiera elegido InferenceClient mediante la API de Hugging Face:

- Hubiera necesitado un plan de pago para saltear el error **Error HTTP 402 (Payment Required)**. El nuevo router de inferencia de Hugging Face exige comprar créditos prepagos o contar con suscripción PRO para consultar Inference Providers.
- Tendría que cambiar SELECTRA (Recognai/zeroshot_selectra_medium) por otro modelo porque Hugging Face no tiene proveedores de nube desplegados en su catálogo serverless para dicho modelo.

# El deploy

### ¿Dónde deployaste y por qué ahí?

El deploy lo realicé en **Streamlit Community Cloud**. Me resulta una herramienta simple, rápida, integración transparente con Github, y con los recursos suficientes (1GB RAM) para implementar proyectos de este tipo.

Evalué **Render** también. El deploy requiere algunos pasos adicionales, aunque no implica mayor complejidad. Sin embargo, siempre hablando de los planes gratuitos, ofrece límites de memoria y límites de uso menores, como así también la función de sleep automático es de pocos minutos (en **Streamlit Community Cloud** permite hasta 12 horas de espera sin entrar en suspensión).

### ¿Por qué tu app necesita un servidor que quede encendido?

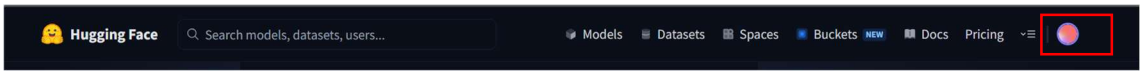
Necesita un servidor encendido continuamente porque:

- Mantiene una conexión bidireccional continua mediante WebSockets entre el navegador del operario y el servidor para transmitir eventos en tiempo real sin recargar la página.
- El modelo SELECTRA reside en la memoria RAM del servidor. Mantener el proceso sostiene el modelo en memoria para responder inmediatamente a cada duda.

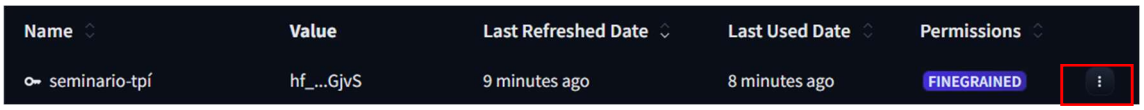
# Lo que se rompió

**Documentá al menos dos errores reales**

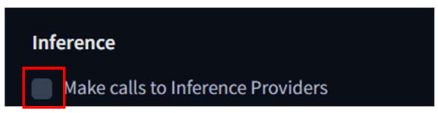
- **Mensaje de error:** 403 Forbidden: This authentication method does not have sufficient permissions to call Inference Providers on behalf of user IgnacioToto27.
- **¿Dónde apareció?**: En mi notebook al intentar probar los modelos de Hugging Face.
- **¿Qué lo causaba?**: En error en la configuración del token de Hugging Face.
- **¿Cómo lo resolviste?**
  - Pasos:
    1. Iniciar sesión en **Hugging Face** https://huggingface.co/settings/tokens
    2. Hacer click en la foto de perfil de usuario




    3. En el menú, ingresar en **Access Token** https://huggingface.co/settings/tokens
    4. Sobre el token del proyecto, desplegar el menú haciendo click en y presionar **Edit**




    5. Activar la opción **Make call to Inference Providers** y guardar.




- **Mensaje de Error:** You have no remaining credits. Purchase pre-paid credits to continue using Inference Providers. Alternatively, subscribe to PRO to get monthly included credits.

- **¿Dónde apareció?**: En mi notebook al intentar probar los modelos de Hugging Face mediante Inference Client.

- **¿Qué lo causaba?**: La versión gratuita de Hugging Face no permite utilizar los modelos mediante Inference Client.

- **¿Cómo lo resolviste?**
  Para evitar las restricciones y costos de la API externa, se implementó la librería transformers y la función pipeline ('zero-shot-classification'). Dado que SELECTRA tiene un tamaño reducido, se descarga en caché local y se ejecuta directamente sobre la CPU consumiendo menos de 300 MB de memoria RAM, a costo cero y tiempos de respuesta inferiores a 200 milisegundos.

# Límites de tu solución

### ¿qué pasa si la usan 1000 personas a la vez?

Posiblemente colapse el contenedor de Streamlit por saturación de CPU y concurrencia. Al procesar inferencias de manera sincrónica en el mismo hilo de ejecución, las peticiones se encolarían generando tiempos de respuesta inaceptables.

### ¿Si Hugging Face cambia las condiciones del plan gratuito (como ya pasó con los Spaces)?

Es exactamente lo que ocurrió durante el desarrollo de este trabajo. Hugging Face comenzó a rechazar peticiones con errores 402 (Payment Required).

### ¿Qué cambiarías para que sea un producto real?

- Utilizaría plataformas de pago que garanticen alta disponibilidad, seguridad y escalabilidad para un universo estimado de 1500 usuarios.
- Implementaría un mecanismo de inicio de sesión con usuario y contraseña y perfiles de acceso con diferentes permisos.
- A nivel código fuente, separar el front-end del backend.
- La app debería estar preparada para ser utilizada en diferentes dispositivos, incluso en entornos Android (para acceder directamente desde la PDA utilizada en el almacén).
- Analizaría mecanismos de integración o envío de dudas en forma automática a los canales a los que el modelo determine como el correcto para atender la consulta del usuario.
- Probar otros modelos pre-entrenados a fin de mejorar la precisión de las respuestas, o entrenar un modelo a fin de optimizar el funcionamiento.

# Vocabulario

```python
class WmsClassifierService:

    """Servicio de negocio para clasificar dudas operativas del WMS."""

    def __init__(self, model_name: str = "Recognai/zeroshot_selectra_medium") -> None:

        from transformers import pipeline

        self.model_name: str = model_name

        self.classifier = pipeline(

            task="zero-shot-classification",

            model=self.model_name,
```

```
            device=-1  # Ejecución en CPU para compatibilidad en PaaS
        )
    def classify_ticket(self, query_text: str, candidate_areas: list[str]) -> dict:
        """Clasifica una consulta y retorna probabilidades ordenadas."""
        hypothesis: str = "Esta consulta corresponde al área de {}."
        prediction_result = self.classifier(
            query_text,
            candidate_labels=candidate_areas,
            hypothesis_template=hypothesis
        )
        return prediction_result
[…]
with st.status("🤖 Procesando consulta mediante NLP...", expanded=True) as status:
            st.write("Cargando modelo en memoria y tokenizando texto...")
            classifier_service = load_classifier_service()
            time.sleep(0.4)
            st.write("Analizando intenciones y contextos operativos...")
            t_inicio = time.time()
            resultado = classifier_service.classify_ticket(consulta_usuario, areas_activas)
```

**Señalización de elementos requeridos**

**Clase**: WmsClassifierService

**Instancia**: classifier_service

**Método**: classify_ticket

**Atributo**: self.model_name

**Parámetro**: query_text

**Argumento**: consulta_usuario, áreas_activas

**Valor de retorno**: prediction_result